Guesmi Mohammed Amine Razi Yassine Chauchet Mattéo

Dossier de Conception UML *Projet APO : Moteur de Jeux de Société au Tour par Tour*

Diagramme de Structure (Moteur de Jeu)

Le diagramme de classes ci-contre présente l'architecture statique du moteur de jeu générique. Il met en évidence les relations structurelles fortes qui unissent les composants fondamentaux du système (Jeu, Plateau, Joueur, Case).

Les choix de cardinalités reflètent une gestion rigoureuse du cycle de vie des objets :

- Gestion des Joueurs (Composition 1 vers 2) : La classe Jeu est liée aux instances JOUEUR_BLANC et JOUEUR_NOIR par une relation de composition forte. La cardinalité est fixée à 2 car ces entités sont instanciées directement par le moteur et n'ont pas d'existence autonome en dehors d'une partie.
- Structure du Plateau (Composition 1 vers *) : Le Plateau est composé d'une multitude de Case (stockées dans des tableaux 2D ou 3D). La cardinalité * associée à une composition indique que la destruction du plateau entraîne systématiquement la destruction de toutes ses cases, garantissant l'intégrité mémoire du modèle.
- Flexibilité des Coups (Relation 0..1 vers Case) : L'objet Coup a été conçu pour supporter tous les types de jeux. La relation vers la case de départ est notée 0..1 car elle est optionnelle : elle est requise pour les jeux de déplacement (Échecs) mais vaut null pour les jeux de placement (Tic-Tac-Toe, Puissance 4), tandis que la case d'arrivée est toujours requise (1).

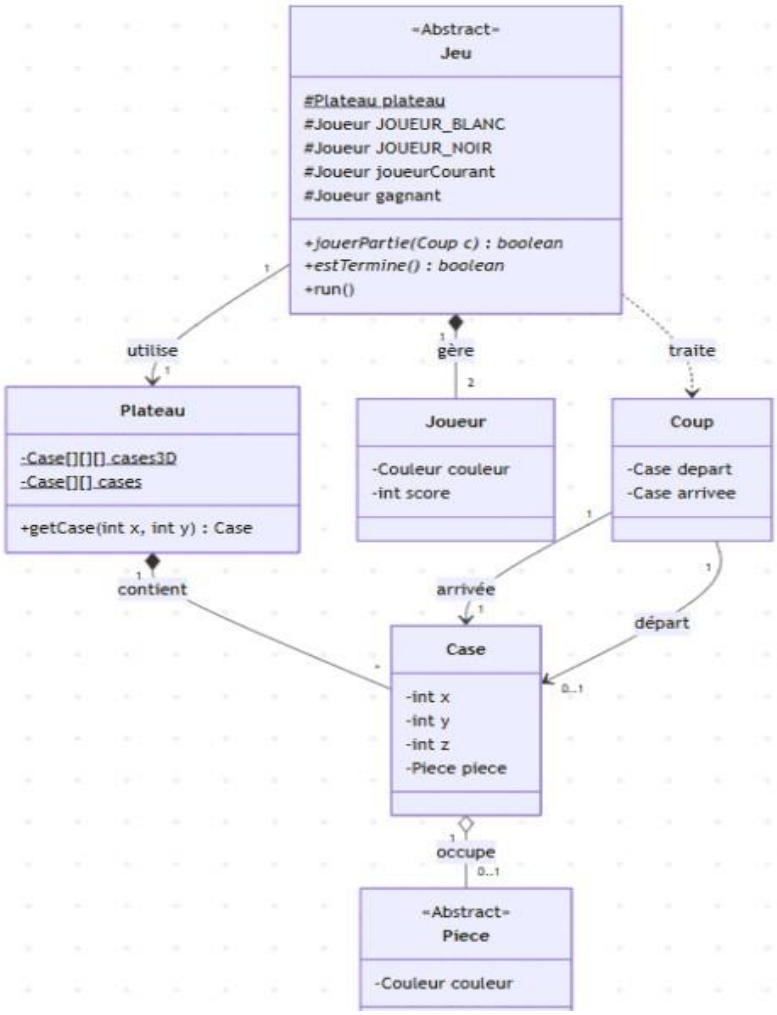

«Abstract»
Jeu
#Plateau plateau
#Joueur JOUEUR_BLANC
#Joueur JOUEUR_NOIR
#Joueur joueurCourant
#Joueur gagnant
+jouerPartie(Coup c) : boolean
+estTermine() : boolean
+run()
utilise
gère
traite
Plateau
-Case[][][] cases3D
-Case[][] cases
+getCase(int x, int y) : Case
Joueur
-Couleur couleur
-int score
Coup
-Case depart
-Case arrivee
contient
arrivée
départ
Case
-int x
-int y
-int z
-Piece piece
occupe
«Abstract»
Piece
-Couleur couleur

Diagramme des Déplacements (Pattern Décorateur)

Le diagramme de classes ci-dessous détaille l'architecture modulaire adoptée pour la gestion des mouvements des pièces. Afin d'éviter la duplication de code et de permettre la création de pièces complexes par composition (ex: Reine = Tour + Fou), nous avons mis en œuvre le Pattern Décorateur.

La cardinalité 0..1 de la relation d'agrégation réflexive sur la classe abstraite DecCasesAccessibles est la clé de voûte de cette conception :

- Composition Dynamique (Relation 0..1) : L'attribut base représente le composant décoré. La cardinalité est définie à 0..1 (et non 1) pour permettre la terminaison de la chaîne de responsabilité. En effet, lors de l'exécution récursive du calcul des cases accessibles, le dernier maillon de la chaîne (la feuille) ne décore aucun autre objet (base = null). Cette modélisation garantit l'arrêt de la récursivité tout en offrant une flexibilité totale pour empiler dynamiquement les comportements de déplacement.

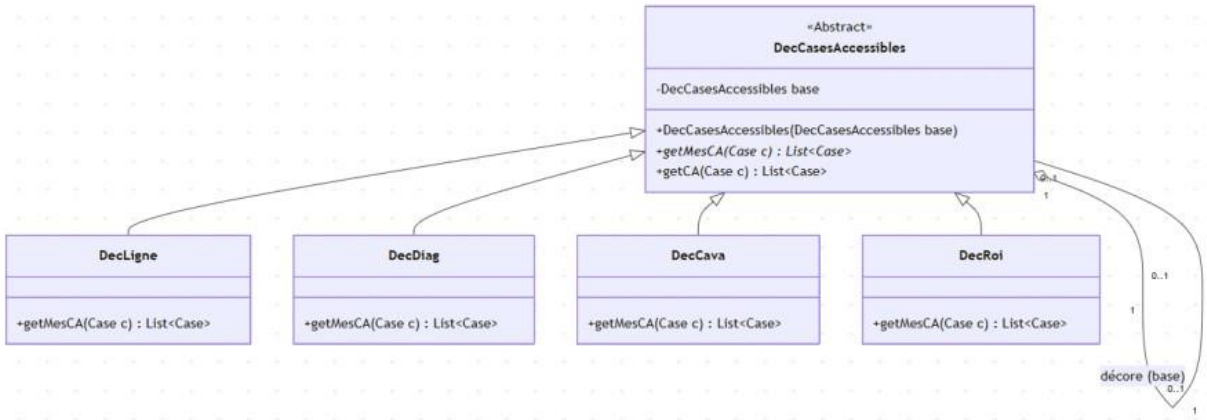
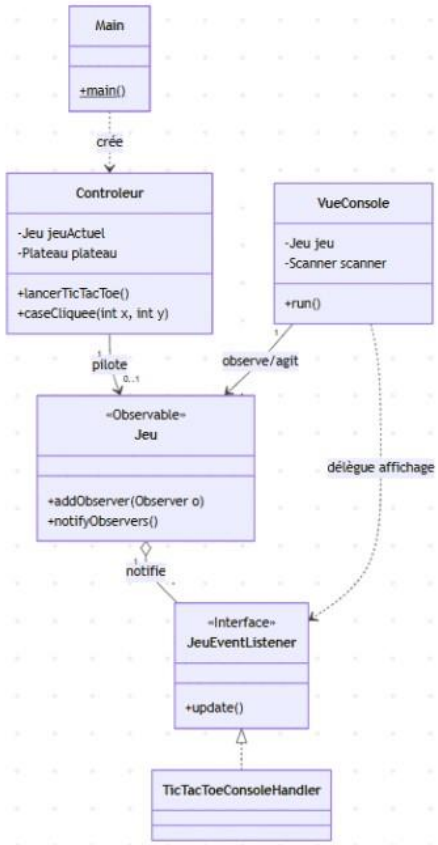


Diagramme MVC & Observer (Architecture)

Le diagramme de classes suivant illustre l'architecture globale de l'application, structurée selon le patron de conception Modèle-Vue-Contrôleur (MVC). Cette séparation garantit une indépendance entre la logique métier (Jeu) et l'interface utilisateur (VueConsole).

Une attention particulière a été portée à la précision des cardinalités pour refléter fidèlement le comportement dynamique et l'extensibilité du système :

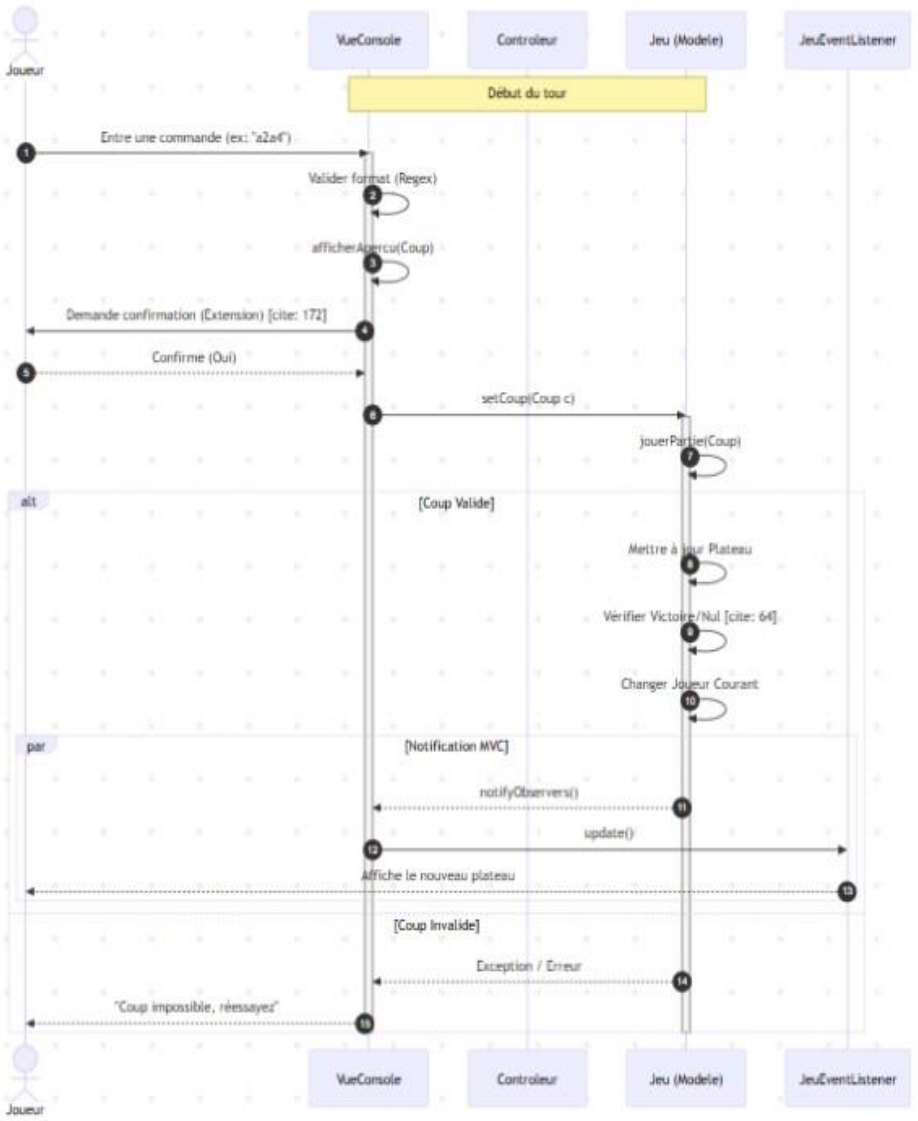
- Extensibilité de la Vue (Relation 1 vers *) : Bien que l'implémentation actuelle se limite à une vue console, l'utilisation du pattern Observer (via l'interface JeuEventListener) justifie une cardinalité multiple (*). Cela démontre que l'architecture est prête à accueillir de nouvelles vues (ex: une interface graphique Swing ou Web) sans nécessiter de modification du Modèle, respectant ainsi le principe de fermeture/ouverture (Open/Closed Principle).
- Cycle de vie du Jeu (Relation 0..1) : La relation entre le Controleur et le Jeu est notée 0..1. Cette cardinalité s'explique par le cycle de vie de l'application : au lancement (dans le menu principal), aucun jeu n'est instancié (null). L'instance de Jeu n'est créée et associée au contrôleur (1) que lorsque l'utilisateur sélectionne une partie spécifique.



Dynamique d'un tour de jeu (Séquence)

Si le diagramme de classes présente la structure statique de notre projet, le diagramme de séquence suivant illustre sa dynamique temporelle. Il détaille le déroulement précis d'un tour de jeu standard, mettant en lumière l'orchestration entre les différentes couches de notre architecture MVC (Modèle-Vue-Contrôleur).

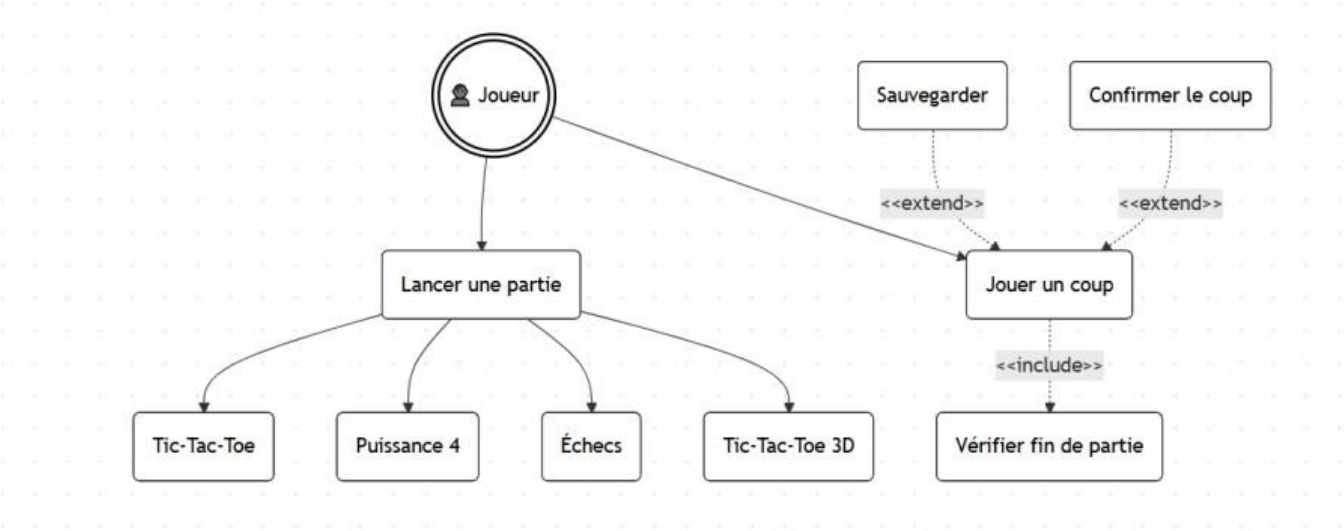
Ce diagramme montre comment une commande utilisateur brute (saisie console) est capturée par la Vue, transmise au Contrôleur pour interprétation, puis traitée par le Modèle (Jeu). Il explicite notamment la boucle de rétroaction (Game Loop) : validation des règles métier via les décorateurs, mise à jour de l'état du plateau, et notification automatique de la vue pour l'affichage du nouvel état.



Analyse fonctionnelle et Cas d'Utilisation

Afin de délimiter le périmètre fonctionnel de notre moteur de jeu, nous avons modélisé les interactions entre l'utilisateur et le système à travers un diagramme de cas d'utilisation. Ce diagramme identifie l'acteur principal (le Joueur) et l'ensemble des fonctionnalités offertes par l'application.

Il met en évidence la modularité de notre approche : le cas d'utilisation générique 'Lancer une partie' se décline en plusieurs variantes (Tic-Tac-Toe, Puissance 4, Échecs, 3D) grâce au mécanisme de généralisation. De plus, conformément au cahier des charges, nous avons intégré les fonctionnalités transverses sous forme d'inclusions (vérification systématique de la fin de partie) et d'extensions (sauvegarde, confirmation de coup), activables à la demande.



1. Cas d'utilisation : Lancer une partie

- Acteur principal : Joueur.
- Description : Le joueur sélectionne l'un des jeux disponibles via le menu principal (Tic-Tac-Toe, Puissance 4, Échecs ou Tic-Tac-Toe 3D).
- Flot d'événements :
    1. Le système affiche le menu principal.
    2. Le joueur choisit une variante.
    3. Le Controleur instancie le Jeu spécifique (Factory Pattern implicite) et le Plateau correspondant (2D ou 3D).
    4. La partie démarre et le premier état du plateau est affiché.
- Lien avec le code : Méthodes lancerTicTacToe(), lancerPuissance4(), etc. dans la classe Controleur.

2. Cas d'utilisation : Jouer un coup

- Acteur principal : Joueur.
- Description : Le joueur effectue une action pour modifier l'état du jeu à son tour.
- Flot d'événements :
    1. Le système invite le joueur courant à saisir une commande (ex: "a2a4" aux échecs ou "1-9" au Tic-Tac-Toe).
    2. Le système analyse la syntaxe de la commande (Parsing).
    3. Le système vérifie la validité sémantique du coup via DecCasesAccessibles (ex: la case est-elle libre ? Le mouvement est-il autorisé pour cette pièce ?).
    4. Si le coup est valide, le pion est placé ou déplacé. Sinon, un message d'erreur invite à recommencer.
- Inclusions : Ce cas inclut systématiquement "Vérifier fin de partie".
- Extensions : Ce cas peut être étendu par "Confirmer le coup" et "Sauvegarder".

3. Cas d'utilisation : Vérifier fin de partie (<< include >>)

- Description : Processus automatique déclenché après chaque coup valide pour évaluer l'état du jeu.
- Logique : Le moteur vérifie si les conditions de victoire (alignement, échec et mat) sont remplies ou si le plateau est plein (match nul).
- Résultat : Si la partie est terminée, le gagnant est notifié et le score est mis à jour. Sinon, le tour passe au joueur suivant.

4. Cas d'utilisation : Confirmer le coup (<< extend >>)

- Contexte : Extension fonctionnelle activable pour éviter les erreurs de saisie.
- Description : Intervient après la saisie d'un coup valide mais avant sa validation définitive.
- Flot :
    1. Le système affiche un aperçu du plateau avec le coup simulé (marqué par des chevrons >X<).
    2. Le joueur valide (O/N).
    3. Si "Non", le coup est annulé et le joueur doit rejouer.

5. Cas d'utilisation : Sauvegarder (<< extend >>)

- Contexte : Extension permettant d'interrompre une partie pour la reprendre plus tard.
- Description : À tout moment lors de la saisie d'un coup, le joueur peut entrer une commande spécifique (ex: "/save") pour sérialiser l'état actuel du Jeu et du Plateau dans un fichier persistant.